Computación y estructuras discretas II

Unidad Programación Funcional: Listas y Recursión Estructural

Juan Marcos Caicedo Mejía – jmcaicedo@icesi.edu.co

Departamento de Computación y Sistemas Inteligentes
Facultad Barberi de Ingeniería, Diseño y Ciencias Aplicadas
Universidad ICESI

Tabla de contenido

1. Listas y Funciones sobre listas

2. Reconocimiento de patrones

3. Ejercicios

4. Más de Scala

5. Más de listas

Contenido

1. Listas y Funciones sobre listas

2. Reconocimiento de patrones

3. Ejercicios

4. Más de Scala

5. Más de listas

Listas

- La lista es una estructura de datos fundamental para la programación funcional.
- La lista que tiene los elementos $x_1, \ldots, x_n$ se escribe en Scala como `List(`$x_1, \ldots, x_n$`)`
- Ejemplos:

```
def minions = List("Kevin", "Dave", "Bob")
def numeros = List(1, 2, 3, 4)
def IDE1 = List(List(1, 0, 0), List(0, 1, 0), List(0, 0, 1))
def vacia = List()
```

- Dos características importantes de las listas: son inmutables y son recursivas.

Los constructores de listas

- Las listas se construyen a partir de dos constructores:
  - La lista vacía `Nil`, y
  - El operador de construcción `::` (se pronuncia *cons*):

$$x :: xs$$

    construye una lista cuyo primer elemento es x, y cuyo resto de elementos es la lista xs.
- Ejemplos:

El tipo Lista

- Las listas son homogéneas: todos sus elementos deben ser del mismo tipo.
- El tipo de una lista de elementos del tipo *T* es `scala.List[`*T*`]` o, en versión corta, `List[`*T*`]`.
- Ejemplos:

Asociatividad a la derecha

- Por convención, los operadores que terminan en ":" son asociativos a derecha, es decir:

$$A :: B :: C \text{ se interpreta como } A :: (B :: C)$$

Por ejemplo:

```
scala> val numeros = 1 :: 2 :: 3 :: 4 :: Nil
val numeros: List[Int] = List(1, 2, 3, 4)
```

es lo mismo que escribir:

```
scala> val numeros = 1 :: (2 :: (3 :: (4 :: Nil)))
val numeros: List[Int] = List(1, 2, 3, 4)
```

Operaciones sobre listas

- Todas las operaciones sobre las listas, se implementan a partir de los constructores de listas y de las siguientes operaciones selectoras:
  - head: devuelve el primer elemento de una lista
  - tail: devuelve la lista sin el primer elemento (el *resto* de la lista)
  - isEmpty: devuelve `true` si la lista es vacía y `false` si no.
- Todas estas operaciones son métodos de los objetos de tipo `List`:

```
scala> minions.head
val res6: String = Kevin

scala> minions.tail
val res7: List[String] = List(Dave, Bob)

scala> minions.tail.head
val res8: String = Dave

scala> minions.tail.tail.head
val res9: String = Bob

scala> minions.tail.tail.tail.head
java.util.NoSuchElementException: head of empty list

scala> minions.tail.tail.tail.tail
java.lang.UnsupportedOperationException: tail of empty list
```

Contenido

1. Listas y Funciones sobre listas

2. Reconocimiento de patrones

3. Ejercicios

4. Más de Scala

5. Más de listas

Reconocimiento de patrones

- El reconocimiento de patrones es una generalización del *switch* de C/Java, para jerarquías de clases. En Scala, se usa a través de la expresión `match`:

```
import scala.util.Random
val x: Int = Random.nextInt(10)
x match {
  case 0 => "zero"
  case 1 => "one"
  case 2 => "two"
  case _ => "other"
}
```

- `match` va precedido por una expresión e y seguido por una secuencia de casos, sintácticamente escritos de la forma `case <pat> => <expr>`.
- Cada caso asocia un patrón `<pat>` con una expresión `<expr>`. Si el resultado de evaluar e cumple el patrón `<pat>`, se devuelve el resultado de `<expr>`.
- Si el resultado de evaluar e no cumple con ningún patrón, se lanzará una excepción `MatchError`.

Evaluación de expresiones `match`

- Una expresión de la forma:

$$\mathtt{e}\ \mathtt{match}\ \{\ \mathtt{case}\ p_1 \Rightarrow \mathtt{e}_1\ \ldots\ \mathtt{case}\ p_n \Rightarrow \mathtt{e}_n\ \}$$

  busca hacer corresponder el valor de la evaluación de e con alguno de los patrones $p_1, \ldots, p_n$ en el orden en que aparecen.

Evaluación de expresiones `match`

- Una expresión de la forma:

$$\mathtt{e}\ \mathtt{match}\ \{\ \mathtt{case}\ p_1 \Rightarrow \mathtt{e}_1\ \ldots\ \mathtt{case}\ p_n \Rightarrow \mathtt{e}_n\ \}$$

  busca hacer corresponder el valor de la evaluación de e con alguno de los patrones $p_1, \ldots, p_n$ en el orden en que aparecen.

- Digamos que p_k es el primer patrón que corresponde con la evaluación de e. La expresión completa se substituye por $\mathtt{e}_k$, haciendo las substituciones a que haya lugar a causa de esa correspondencia.

Evaluación de expresiones `match`

- Una expresión de la forma:

$$\mathtt{e\ match}\,\{\,\mathtt{case}\,p_1 \Rightarrow \mathtt{e}_1\ \ldots\ \mathtt{case}\,p_n \Rightarrow \mathtt{e}_n\,\}$$

  busca hacer corresponder el valor de la evaluación de e con alguno de los patrones $p_1, \ldots, p_n$ en el orden en que aparecen.
- Digamos que p_k es el primer patrón que corresponde con la evaluación de e. La expresión completa se substituye por $\mathtt{e}_k$, haciendo las substituciones a que haya lugar a causa de esa correspondencia.
- Específicamente, las referencias a variables del patrón dentro de $\mathtt{e}_k$ se substituyen por los valores con que se logró la correspondencia.

Patrones de listas

- Es posible descomponer las listas con reconocimiento de patrones:

Patrones de listas

- Es posible descomponer las listas con reconocimiento de patrones:
  - `Nil`: el patrón constante `Nil`.

Patrones de listas

- Es posible descomponer las listas con reconocimiento de patrones:
  - `Nil`: el patrón constante `Nil`.
  - `p :: ps`: un patrón que hace corresponder la cabeza de la lista con el patrón *p* y la cola de la lista con el patrón *ps*. Este patrón, por ejemplo, no hace correspondencia con una lista vacía.

Patrones de listas

- Es posible descomponer las listas con reconocimiento de patrones:
  - `Nil`: el patrón constante `Nil`.
  - p :: ps: un patrón que hace corresponder la cabeza de la lista con el patrón p y la cola de la lista con el patrón ps. Este patrón, por ejemplo, no hace correspondencia con una lista vacía.
  - List($p_1, \ldots, p_n$): es una abreviación del patrón

$$p_1 :: p_2 :: \cdots :: p_n :: \texttt{Nil}$$

Contenido

1. Listas y Funciones sobre listas

2. Reconocimiento de patrones

3. Ejercicios

4. Más de Scala

5. Más de listas

Ejercicios

1. Implementar en Scala la función que calcule la suma de todos los elementos de una lista de enteros.
2. Implementar en Scala la función que agregue un elemento en la última posición de una lista de enteros.
3. Implementar en Scala una función que retorne el iésimo elemento de una lista.
4. Programe en Scala una función recursiva que recibe como entrada una lista que contiene *N* listas de números. El programa retorna la lista que contiene los *N* números que resultan al sumar los números en cada una de las listas de la entrada.

Contenido

1. Listas y Funciones sobre listas

2. Reconocimiento de patrones

3. Ejercicios

4. Más de Scala

5. Más de listas

Documentación de Scala asociada a esta ses

- Enlace a la documentación de colecciones en Scala
- Enlace a la documentación de Traits
- Enlace a la documentación de Clases case, enlace a la documentación de pattern matching

Contenido

1. Listas y Funciones sobre listas

2. Reconocimiento de patrones

3. Ejercicios

4. Más de Scala

5. Más de listas

Métodos predefinidos (1)

Métodos para acceder a elementos y sublistas:

- `xs.length`: número de elementos de `xs`

Métodos predefinidos (1)

Métodos para acceder a elementos y sublistas:

- `xs.length`: número de elementos de `xs`
- `xs.last`: el último elemento de `xs`; lanza excepción si `xs` es vacía

Métodos predefinidos (1)

Métodos para acceder a elementos y sublistas:

- `xs.length`: número de elementos de `xs`
- `xs.last`: el último elemento de `xs`; lanza excepción si `xs` es vacía
- `xs.init`: una lista con los mismos elementos de `xs` salvo el último; lanza excepción si `xs` es vacía

Métodos predefinidos (1)

Métodos para acceder a elementos y sublistas:

- `xs.length`: número de elementos de `xs`
- `xs.last`: el último elemento de `xs`; lanza excepción si `xs` es vacía
- `xs.init`: una lista con los mismos elementos de `xs` salvo el último; lanza excepción si `xs` es vacía
- `xs take n`: una lista con los primeros `n` elementos de `xs` (o `xs` si tiene menos de `n` elementos)

Métodos predefinidos (1)

Métodos para acceder a elementos y sublistas:

- `xs.length`: número de elementos de `xs`
- `xs.last`: el último elemento de `xs`; lanza excepción si `xs` es vacía
- `xs.init`: una lista con los mismos elementos de `xs` salvo el último; lanza excepción si `xs` es vacía
- `xs take n`: una lista con los primeros `n` elementos de `xs` (o `xs` si tiene menos de `n` elementos)
- `xs drop n`: el resto de la lista `xs` después de quitar los primeros `n` elementos

Métodos predefinidos (1)

Métodos para acceder a elementos y sublistas:

- `xs.length`: número de elementos de `xs`
- `xs.last`: el último elemento de `xs`; lanza excepción si `xs` es vacía
- `xs.init`: una lista con los mismos elementos de `xs` salvo el último; lanza excepción si `xs` es vacía
- `xs take n`: una lista con los primeros `n` elementos de `xs` (o `xs` si tiene menos de `n` elementos)
- `xs drop n`: el resto de la lista `xs` después de quitar los primeros `n` elementos
- `xs(n)`: el `n`-ésimo elemento de `xs`; lanza excepción si `xs` no tiene al menos `n` elementos

Métodos predefinidos (2)

- Creación de nuevas listas:
  - `xs ++ ys`: la lista que consiste de todos los elementos de `xs` seguidos de todos los elementos de `ys` (concatenación)
  - `xs.reverse`: la lista con los mismos elementos de `xs` pero en orden inverso
  - `xs updated (n,x)`: una lista con los mismos elementos de `xs` salvo el *n*-ésimo que ahora es `x`; lanza excepción si `xs` no tiene al menos `n` elementos

Métodos predefinidos (2)

- Creación de nuevas listas:
  - `xs ++ ys`: la lista que consiste de todos los elementos de `xs` seguidos de todos los elementos de `ys` (concatenación)
  - `xs.reverse`: la lista con los mismos elementos de `xs` pero en orden inverso
  - `xs updated (n,x)`: una lista con los mismos elementos de `xs` salvo el *n*-ésimo que ahora es `x`; lanza excepción si `xs` no tiene al menos `n` elementos
- Métodos para buscar elementos:
  - `xs indexOf x`: devuelve el índice del primer elemento en `xs` igual a `x`; si no aparece `x` en `xs` devuelve -1
  - `xs contains x`: devuelve (`xs indexOf x >= 0`)

Implementando funciones sobre listas: *last* e *init*

- La complejidad de *head* es constante. ¿Qué se puede decir de la complejidad de *last*?
- Miremos una posible implementación de *last*:

```
def ultimo[T](xs: List[T]): T = xs match {
  case List() => throw new Error("ultimo_de_una_lista_vacia")
  case List(x) => x
  case _ => ultimo(xs.tail)
}
```

Su complejidad es proporcional al tamaño de `xs`

- Cómo implementaría *init*:

```
def init[T](xs: List[T]): T = xs match {
  case List() => throw new Error("
      listainicial_de_una_lista_vacia")
  case List(x) => ???
  case _ => ???
}
```

Implementando funciones sobre listas: *concat* y *reverse*

- ¿Cómo podríamos implementar la concatenación?

```
def concat[T](xs:List[T], ys:List[T]): List[T] = xs match {
  case List() => ys
  case x::zs => x::concat(zs,ys)
}
```

¿Cuál es su complejidad?

- Cómo implementaría *reverse*:

```
def reverse[T](xs:List[T]):List[T] = xs match {
  case List() => xs
  case List(x) => xs
  case x::ys => reverse(ys) ++ List(x)
}
```

¿Cuál es su complejidad? ¿Se podría mejorar?

Implementando funciones sobre listas: *aplanar*

- Las listas no tienen que ser homogéneas. Pueden tener elementos de diferentes tipos mezclados:

```
scala> 2::List("a","b")
val res0: List[Any] = List(2, a, b)

scala> List(2, List("Dave", "Bob"), "nissan")
val res1: List[Any] = List(2, List(Dave, Bob), nissan)
```

- Una función útil cuando se tienen listas de listas, es la que aplana la lista, es decir, devuelve la lista con los elementos básicos que no son listas:

```
def aplanar(xs:List[Any]): List[Any] = xs match {
  case List() => xs
  case y::ys => y match {
    case List() => aplanar(ys)
    case z::zs => aplanar(z::zs) ++ aplanar(ys)
    case _ => y :: aplanar(ys)
  }
}

aplanar(List(List(1,1), 2, List(3, List(5,8))))
```